Multi Client LAN Chat & File Sharing System

Project Overview

The proposed project is a multi-client communication system built using Python networking technologies.

The system follows a Client-Server Architecture, Where one Python application acts as the central server and multiple clients connect to it over a Local Area Network (LAN).

Each client represents a user who can log in, view online users, send private message, participate in group conversations, and exchange files with other connected users.

The server is responsible for managing all client connections, routing messages, controlling group communications, and coordinating file transfers.

الفكرة ببساطة اننا هنعمل برنامج Communication System شبيه بشكل مبسط بتطبيقات ال Chat لكن الهدف الاساسي هنا تطبيق مفاهيم ال Network Programming .

هيكون عندنا:

- Server → Central Server
- Clients → User Devices
- LAN → الشبكة الي الاجهزة متصلة عليها

كل client يتصل بال server وال server هو الي ينظم الاتصال بينهم.

ex : Menna → Server → Ahmed

يعني Menna مش بتبعت مباشرة ل Ahmed.

الرسالة بتروح للسيرفير والسيرفر يعرف ان Ahmed موجود فيعمل لها routing ويوصلها له.

Problem Statement

In a local environment such as a university laboratory, office, or organization, users may need to communicate and exchange file quickly without depending on external communication platforms.

The project aims to provide a simple centralized communication system that allows multiple users on the same LAN to communicate and Transfer files through a dedicated server.

The project also demonstrates practical networking concepts such as TCP communication, socket programming , concurrent connections , message routing and file transfer.

ال Target اننا نعمل Local Communication system يشتغل داخل شبكة معينة .

ex:

- University Lab
- Office
- Small Organization
- Computer Lab

ال Users يقدروا ينقلوا ال Files من خلال السيرفير الموجود داخل الشبكة.

Project Objectives

The main objectives of the project are:

1. Build a real client-server network application.
2. support multiple clients simultaneously.
3. Implement TCP socket communication.
4. Implement user authentication.
5. Manage online and offline users.
6. Support private messaging.
7. Support group communication.
8. Support file transfer between users.
9. Display file transfer progress.
10. Build a simple graphical user interface.
11. Store selected user and communication data using a database.
12. Demonstrate the system over a real LAN.

Project Solution

The proposed solution is centralized communication platform consisting of:

- A python based server.
- Multiple python based client.
- A TCP communication layer.
- A custom application layer communication protocol.
- A graphical user interface.
- Optional SQLite database storage.

The server will remain active and listen for incoming client connections . Each connected client will communicate with the server through a TCP socket.

Client

↓

TCP Connection

↓

Server

↓

TCP Connection

↓

Another Client

ال Server بيكون شغال ومستني Clients تعمل Connection.

كل Client بيعمل TCP Socket Connection مع ال Server.

System Architecture

The system will use a centralized Client-server Architecture.

SERVER

Python server.py

IP + Port 5000

|

↓ ↓ ↓

Client 1 Client 2 Client 3

مش هنستخدم Peer to Peer architecture.

ال central point هو ال Server.

client to عايز يبعت ل client 1

Client 1

↓

Server

↓

client 2

وده ب allow ال serverيتحكم في :

- مين online
- مين offline
- مين يبعت لمين
- ال Groups

- File Transfers
- Authentication

Server

The server is a python application responsible for managing the entire communication system.

Its main responsibilities include:

- Accepting client connections.
- Authenticating users.
- Managing active connections.
- Tracking online users.

- Routing private messages.
- Managing group communication.
- Coordinating file transfers.
- Handling client disconnections.
- Logging important system events.

Python program :

server.py

وظيفته إنه يكون العقل اللي بينظم الـ system.

مثلا لو 3 clients متصلين :

Server
├── Client 1
├── Client 2
└── Client 3

السيرفر يعرف كل واحد متصل، ويعرف كل Connection تخص مين .

Client

A client is the application used by an individual user to communicate with the server.

Each client establishes a TCP connection with the server.

The client provides the graphical interface through which the user can:

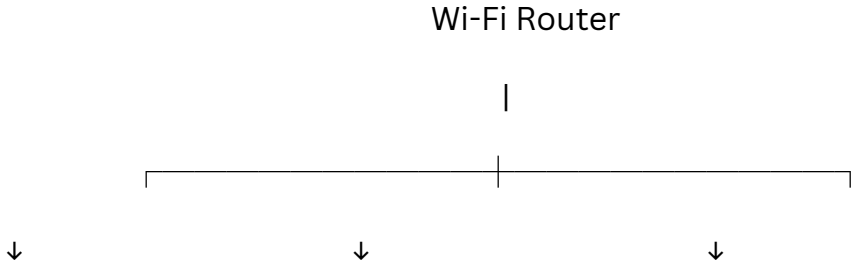
- Log in.
- View online users.
- Send messages.
- Receive messages.
- Join groups.
- Send files.
- Receive files.

client.py

Local Area Network (LAN)

The system is initially designed to operate over a Local Area Network.

For example, multiple laptops can connect to the same Wi-Fi network.



Server client 1 client 2

لو عندنا 3 Laptops

- Laptop 1 = Server
- Laptop 2 = Client 1
- Laptop 3 = Client 2

والتلاتة متصلين بنفس ال wi-fi

ال سثقرثق مثلا يكون 192.168.1.5

وال client يتصل ب 192.168.1.5:5000

ال 5000 هنا ال port الي السرفير بيستمع عليه.

TCP Socket Communication

The system will use **TCP sockets** as the main communication mechanism.

TCP provides reliable, ordered, connection-oriented communication between the clients and the server.

The server will create a socket, bind it to an IP address and port, listen for incoming connections, and accept clients.

ال Socket هو الوسيلة الي Python بيستخدمها عشان يعمل network communication.

وال TCP مهم لانه بيضمن ان البيانات:

- توصل
- توصل بالترتيب
- من غير فقدان غير متوقع للبيانات اثناء ال connection

ال server هيعمل

Create Socket
↓
Bind IP +Port
↓
Listen
↓
Accept Client

وال client

Create Socket
↓
Connect to server

Multi Client Communication

The server must support multiple clients at the same time.

A separate thread can be assigned to each connected client so that one client's activity does not block the communication of other clients.

Server
│
├──── Thread → Client 1
├──── Thread → Client 2
└──── Thread → Client 3

لو عندنا:

client 1
client 2
client 3

الثلاثة متصلين في نفس الوقت .
مش عايزين السيرفر يتعامل مع واحد ويقف لحد ما التاني يخلص
علشان كده هنستخدم ال Multithreading
كل client ممكن يكون له thread مسؤول عن التعامل مع ال connection بتاعته

User Authentication

Before accessing the communication features, the user must authenticate using a username and password.

The server validates the credentials and determines whether the user is allowed to connect.

Possible responses include:

LOGIN_SUCCESS

LOGIN_FAILED

ال user مش هيدخل علي البرنامج وخلاص
هيكون عنده :

Username
Password

ال client يبعت Login Request للسيرفر .
ال server يتاكد من ال data
لو صح LOGIN_SUCCESS
لو غلط LOGIN_FAILED

Online Users

The server will maintain a list of currently connected users.

Clients can request the list of online users.

Example:

Online Users

- Client 1
- Client 2
- Client 3

السيرفر عنده list بالناس الي Online حاليا

لما client يطلب : GET_USERS

السيرفر يرجعله المستخدمين المتصلين

وده يسمح المستخدم يختار الشخص الي عايز يكلمه

Private Messaging

Users can send private messages to specific users.

Example:

Sender: Client 1

Receiver: Client 2

Message: Hello Client 2!

The message is sent to the server first. The server identifies the receiver and forwards the message to the appropriate client.

لو client 1 بعت ل client 2

Client 1 → Server → Client 2

not client 1 → client 2

لان ال server هو الي مسؤول عن ال routing

ex of application layer message

{

"type": "SEND_MESSAGE",

"sender": "Client 1",

"receiver": "Client 2",

"message": "Hello Client 2!"

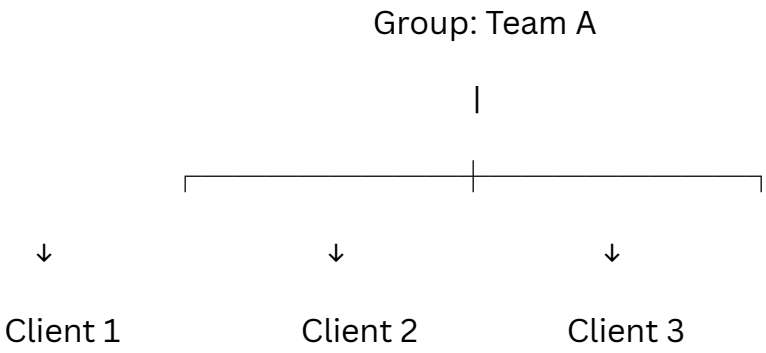
}

Group Chat

The system can support group conversations.

A user can create or join a group, and the server manages the members of each group.

When a user sends a message to a group, the server broadcasts the message to all members of that group.



بدل ما الكلام يكون بين شخصين ممكن نعمل group

ex: Network Programming Team

- client 1
- client 2
- client 3

لو client 1 بعت message في ال group السيرفر يعمل Broadcast لباقي اعضاء ال group

File Sharing

One of the main features of the project is file transfer between connected clients.

The sender selects a file through the GUI.

The client first sends file metadata to the server, such as:

- File name.
- File size.
- Receiver.
- File identifier.

The actual file is then transferred in smaller chunks instead of sending the entire file as one large message.

client 1 عايز يبعت مثلا network_project.pdf ل client 2

مش هنحاول نبعت ال file مرة واحدة

هنبعت الاول metadata

Filename / File Size/ Sender/ Receiver/ File ID

وبعدها الملف نفسه هيتقسم ل

File
│
├── Chunk 1
├── Chunk 2
├── Chunk 3
├── Chunk 4
└── ...

File Transfer Flow

The file transfer process can follow this sequence:

Sender
↓
Select File
↓
Send File Metadata
↓
Server
↓
Notify Receiver
↓
Receiver Accepts
↓
File Chunks Transfer
↓
Receiver Reconstructs File
↓
Transfer Complete
100%

ال workflow

menna تختار الفايل

إل client يبعت معلومات الملف للسيرفر .

السيرفر يعرف ان menna عايزة تبعته لـ ahmed

السيرفر يبلغ ahmed

يوافق ahmed

يبدا نقل الملف

الملف يتبعت chunks

chunk يجمع ال ahmed

الملف يتكون من جديد

يظهر Transfer Completed 100%

File Transfer Progress

The GUI can display the progress of a file transfer.

Example:

Sending: project.pdf



8.0 MB / 10 MB

بدل ما ال user يضغط send والبرنامج يختفي هيشوف :

20%

40%

60%

80%

100%

وده ممكن يتحسب من عدد ال bytesالي تم ارسالها بالنسبة لحجم الملف الكلي

File Integrity

The system can optionally calculate a checksum, such as SHA-256, before and after transmission.

The receiver can compare the checksum of the received file with the original checksum to verify that the file was transferred correctly.

دي advanced feature ممكن نضيفها بعد ما الاساس يشتغل لو فيه وقت

فكرتها اننا نحسب ال Hash قبل الارسال SHA-256

وبعد ما الملف يوص نحسبه تاني

لو الاتنين متساويين

Original Hash == Received Hash

يبقي الملف غالبا وصل بدون تغيير او corruption

Application Layer Protocol

The clients and server will communicate using a custom application-layer protocol.

Control messages can be represented using JSON.

Example:

```
{
   "type": "SEND_MESSAGE",
   "sender": "Menna",
   "receiver": "Ahmed",
   "message": "Hello!"
}
```

Possible message types include:

LOGIN

LOGIN_SUCCESS

LOGIN_FAILED

LOGOUT

GET_USERS

SEND_MESSAGE

CREATE_GROUP

JOIN_GROUP

SEND_FILE

FILE_ACCEPT

FILE_CHUNK

FILE_COMPLETE

احنا محتاجين طريقة متفق عليها بين ال client وال server عشان يفهموا بعض

يعني لو السيرفر استقبل SEND_MESSAGE

يعرف ان دي chat message

ولو استقبل SEND_FILE

يعرف ان فيه File Transfer

ال JSON مناسب لل Control message لانه سهل القراءة والتعامل معاه في python

لكن ال Binary File Data مش هنحطها كلها جوه JSON هنعمل لها Protocol لنقل ال Chunks

GUI

The final client application will include a graphical user interface.

A possible interface structure is :

Chat Application

Online Users	Chat Area
Ahmed	Ahmed: Hello! Menna: Hi Ahmed!
Jana	Message..... [Send] [File]

ال GUI ممكن يكون فيه

Login Screen.
Online Users.
Chat Area.
Message Box.
Send Button.
Attach File Button.
File Transfer Progress.
Group Chats.
Logout.

ممكن نستخدم Python Thinker بدل ال frontend

Database

SQLite can be used for local persistence.
Possible tables include:

Users

id
username
password_hash
status
last_seen

Messages
id
sender
receiver
message
timestamp

Files
id
sender
receiver
filename
size
timestamp
status

مش شرط اول حاجة نعملها ال Database
الاولوية
Networking → Multi-client → Chat → File Transfer → Database
بعد ما ال core يشتغل نقدر نخزن

Users.
Messages.
File information.
Timestamps.
Transfer status.

Technologies

The main technologies and Python libraries will include:

Technology	Purpose
Python	Main programming language
Socket	Network communication
TCP	Reliable communication
Threading	Multiple client handling
JSON	Control-message format
Tkinter	Graphical User Interface

SQLite	Data persistence
Hashlib	Password hashing / file integrity
OS	File and system operations
Struct	Binary protocol handling

Complete System Workflow

The complete workflow can be summarized as follows:

Start Server
↓
Create Socket
↓
Bind IP + Port
↓
Listen for Connections
↓
Client Starts
↓
Client Connects to Server
↓
Login
↓
Authentication
↓
Load Online Users
↓
Start Communication
↓
Private Chat / Group Chat
↓
File Sharing
↓
File Transfer Progress
↓
Transfer Complete
↓
Logout
↓

Connection Closed

Final Project Structure

A possible final project structure could be:

```
NetworkChat/
│
├─── server/
│    ├─── server.py
│    ├─── client_manager.py
│    ├─── message_handler.py
│    ├─── file_handler.py
│    └─── database.py
│
├─── client/
│    ├─── client.py
│    ├─── gui.py
│    ├─── chat.py
│    └─── file_transfer.py
│
├─── protocol/
│    └─── protocol.py
│
├─── database/
│    └─── chat.db
│
└─── README.md
```